

Assignment 3: IT Workshop I

Total Marks = 10

Question 1 — Positive/Negative Sum Across Test Cases (Marks = 5)

Suppose we want to write a function that calculates the sum of positive and negative numbers present in an array. The array represents one test case for a user. Consider a scenario where a user wants to provide n test cases and calculate the sum of positive and negative numbers for each test case, along with the overall sum of positive and negative numbers considering all test cases.

The user has a restriction that all the test cases must be submitted at one go rather than one by one. Write a Java program to solve the given problem. The user can provide input to the program in three steps as follows:

- Input 1: Value of n , representing the number of test cases.
- Input 2: An array of size n containing the number of elements in each test case.
- Input 3: Values of the n different test cases.

Note: The length of the test cases need not be uniform.

For example, if $n = 3$ and the second input is $\{4, 3, 5\}$, then the first test case contains 4 elements, the second contains 3 elements, and the third contains 5 elements.

Task

For each test case, display the sum of positive numbers and the sum of negative numbers. Finally, display the overall sum of positive and negative numbers across all test cases.

Question 2 — Bank Account Management System (Marks = 5)

Consider a Bank Account Management System and implement the following in Java. The variable/method names indicate their usual meanings. The methods (deposit, withdraw, etc.) will be invoked for one account at a time, and the account will be identified by the account number passed as a parameter or entered by the user.

Account
- accountNo: int - accountHolderName: String - accountType: String - balance: double - branchName: String - minimumBalance: double
<<constructor>> Account() <<constructor>> Account(balance: double) + setDetails() + setDetails(no: int, name: String, type: String, balance: double, branch: String) + getDetails(no: int) + deposit(no: int, amount: double) + withdraw(no: int, amount: double)

a) Array of Objects

Create an array of objects (at least 5) for the Account class and set the details for each account object.

b) Menu-Driven Interface

Design a menu-driven interface for the user. Users will choose an option from:

- 1. Set Details
- 2. Get Details
- 3. Deposit Money
- 4. Withdraw Money
- 5. Exit

Based on the option entered by the user, the appropriate operation should be performed by calling the corresponding method of the class.

c) Extensibility

You can assume additional instance variables, additional methods, static fields, etc., if needed to make the application more realistic.

Note: You can use the following syntax to create an array of objects:

```
Account[] accountArray = new Account[5];  
accountArray[0] = obj1; accountArray[1] = obj2; accountArray[2] = obj3;  
accountArray[3] = obj4; accountArray[4] = obj5;  
where obj1, obj2, obj3, obj4, and obj5 are the Account objects.
```